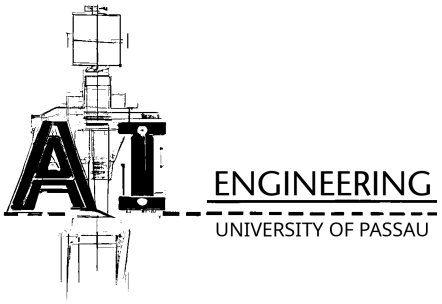
Mining Software Repositories

Exercise 2

Prof. Dr. Steffen Herbold
Winter 2026
Due on 28.10.2026



Author: Lukas Schulte

Task 1: Collecting data from GitHub

1.1 Exploring GitHub as a data source

GitHub is a forge that contains multiple tools, which can provide data of issues, pull requests (PRs) and code reviews. Familiarize yourself with the GitHub repository of the Apache Ignite project.[1]

1. Which tools does the Ignite Project use within GitHub?
2. Focusing on what happens during a PR:
   - In which roles do developers participate in a PR? One example is the author of the PR.
   - What are the different states a PR can be in?
   - What happens to the commits made on the PR branch, once it is merged?

1.2 Mining GitHub with Python

Like Jira in last week's exercise, GitHub can be accessed through a REST API. Unlike Jira though, GitHub employs a rate limit on requests made to that API. Therefore, if we want to make more than a handful of requests, we need to use authentication. For this, you need a GitHub account. Once logged in, follow this link to create a "personal access tokens (classic)":

`https://github.com/settings/tokens`

You can now explore the API by setting the token as a "Bearer Token" in the authorization tab of the Hoppscotch / Postman API tool. Refer to the documentation[2] for information about which endpoints exist.

Prepare a Python development environment and work on the following tasks:

1. Create a Python script that accesses the Apache Ignite repository on GitHub. Note that using the REST API is a requirement. Using the abstract wrapper of a third-party library to make the requests is not the point of this exercise.
2. Iterate over the PRs of the repository and store at least the following information: ID of the PR, title, text, merged status, closed status, opened and closed date, list

[1] Ignite `https://github.com/apache/ignite`
[2] GitHub REST API: `https://docs.github.com/en/rest`

of comments with text and author, list of review comments with text and author, information about changed files.

   - While iterating, make sure you are aware of pagination. Refer to the GitHub REST API documentation for more information.
   - Also while iterating, make sure that your script respects the rate limit. Come up with a solution to wait until request capacity comes available again. Reference the GitHub REST API documentation for more information.[3] (Hint: there are rate limit response headers.)

3. Iterate the issues of the repository and store all those fields, which you deem to be important.

   - How does the data available differ from last weeks Jira data?
   - Make sure to be aware of pagination and to respect the rate limit as well.

4. Make sure to store your data in a format which you can use later for an analysis. Consider using a pandas data frame.

Task 2: Data Analysis

In this task, you will perform some light data analysis. Nothing too complex, just a few graphs to see how your data looks like. Feel free to continue exploring your data beyond what is listed here.

1. Calculate the absolute number and the percentage of how many PRs are merged and how many are closed without merging.
2. Draw a box plot displaying how long the PRs are usually open (time delta between opened and closed). Be prepared to explain what it is one can see looking at the box plot.
3. Draw two more box plots which compare the number of review comments between PRs that have been merged and PRs that were closed without merging. How can you check if there is a significant difference between those sample groups? Refer to the online book 'Introduction to Data Science'[4] to freshen up your knowledge about statistics.

[3] `https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api`
[4] `https://sherbold.github.io/intro-to-data-science/11_Statistics.html`